

UNIDAD 4: MACHINE LEARNING

Contenido

CAPÍTULO 0.- MACHINE LEARNING. INTRODUCCIÓN	1
CAPÍTULO 1.- REGRESIÓN LOGÍSTICA	2
1.1. ¿Qué es la regresión logística?	2
1.2. ¿Por qué es importante la regresión logística?	2
1.3. ¿Cuáles son las aplicaciones de la regresión logística?	3
1.4. ¿Cómo funciona el análisis de regresión?	4
1.5. ¿Cómo funciona el modelo de regresión logística?	5
1.6. ¿Cuáles son los tipos de análisis de regresión logística?	8
1.7. ¿Cómo se compara la regresión logística con otras técnicas de ML?	9
BIBLIOGRAFÍA	13

CAPÍTULO 0.- MACHINE LEARNING. INTRODUCCIÓN

En los temas anteriores trabajamos con **regresión lineal simple** y **regresión lineal múltiple**, aprendiendo a ajustar modelos que explican **una variable continua** a partir de una o varias variables predictoras. Ese fue nuestro primer gran paso dentro del aprendizaje supervisado: entender cómo un modelo puede capturar relaciones lineales y utilizarlas para predecir.

Ahora vamos a **dar un paso más** y adentrarnos en técnicas que amplían ese marco y nos permiten abordar problemas distintos y, en muchos casos, más realistas. En concreto, introduciremos dos herramientas fundamentales en machine learning: la **regresión logística**, que nos permite modelar **variables categóricas** (especialmente problemas de clasificación binaria), y los **árboles de decisión**, modelos flexibles y muy interpretables capaces de capturar **relaciones no lineales y reglas de decisión complejas**.

Ambos métodos representan un avance natural respecto a la regresión lineal: mantienen la idea de aprender a partir de datos, pero amplían el tipo de problemas que podemos resolver y la forma en que los modelos se adaptan a la información disponible. A partir de aquí, empezaremos a ver cómo el aprendizaje automático nos ofrece un abanico mucho más amplio de posibilidades.



CAPÍTULO 1.- REGRESIÓN LOGÍSTICA

1.1. ¿Qué es la regresión logística?

Tal como vimos en los temas anteriores sobre **regresión lineal simple** y **regresión lineal múltiple**, donde modelamos relaciones entre variables continuas, ahora utilizamos el aprendizaje supervisado para abordar problemas de **clasificación**. A continuación, exploraremos el **modelo de regresión logística**, una técnica que permite predecir variables categóricas a partir de características numéricas.

La regresión logística es un algoritmo supervisado que se utiliza para resolver problemas de clasificación. A diferencia de la regresión lineal, cuya salida es continua, la regresión logística produce una salida **discreta**, como "Sí/No" o una clase entre varias posibles.

La **regresión logística** es una técnica de aprendizaje supervisado que permite **predecir variables categóricas a partir de características numéricas**. A diferencia de la regresión lineal, que modela relaciones entre variables continuas, la regresión logística se utiliza para resolver problemas de clasificación, produciendo resultados discretos como "Sí/No" o múltiples clases.

1.1.1. Ejemplos de aplicación:

- Clasificar correos como "Spam" o "No Spam".
- Diagnóstico médico: "Benigno" o "Maligno".
- Clasificación de artículos: "Deportes", "Política", "Ciencia", etc.
- Decisión crediticia: "Conceder crédito" o "No conceder crédito".

1.1.2. Casos prácticos

Caso práctico 1: Clasificación del Sistema Operativo

Por ejemplo, supongamos que desea adivinar si el visitante de su sitio web va a hacer clic en el botón de pago de su carrito de compras o no. El análisis de regresión logística analiza el comportamiento de los visitantes anteriores, como el tiempo que permanecen en el sitio web y la cantidad de artículos que hay en el carrito. Determina que, si anteriormente los visitantes pasaban más de cinco minutos en el sitio y agregaban más de tres artículos al carrito, hacían clic en el botón de pago. Con esta información, la función de regresión logística puede predecir el comportamiento de un nuevo visitante en el sitio web.

1.2. ¿Por qué es importante la regresión logística?

La regresión logística es una técnica importante en el campo de la [inteligencia artificial](#) y el [aprendizaje automático](#) (AI/ML). Los modelos de ML son programas de software que puede entrenar para realizar tareas complejas de procesamiento de datos sin intervención humana. Los modelos de



ML creados mediante regresión logística ayudan a las organizaciones a obtener información procesable a partir de sus datos empresariales. Pueden usar esta información para el análisis predictivo a fin de reducir los costos operativos, aumentar la eficiencia y escalar más rápido. Por ejemplo, las empresas pueden descubrir patrones que mejoran la retención de los empleados o conducen a un diseño de productos más rentable.

A continuación, enumeramos algunos **beneficios del uso de la regresión logística en comparación con otras técnicas de ML**.

- **Simplicidad**

Los modelos de regresión logística son matemáticamente menos complejos que otros métodos de ML. Por lo tanto, puede implementarlos incluso si nadie de su equipo tiene una profunda experiencia en ML.

- **Velocidad**

Los modelos de regresión logística pueden procesar grandes volúmenes de datos a alta velocidad porque requieren menos capacidad computacional, como memoria y potencia de procesamiento. Esto los hace ideales para que las organizaciones que están empezando con proyectos de ML obtengan ganancias rápidas.

- **Flexibilidad**

Puede usar la regresión logística para encontrar respuestas a preguntas que tienen dos o más resultados finitos. También puede usarlo para preprocesar datos. Por ejemplo, puede ordenar los datos con un amplio rango de valores, como las transacciones bancarias, en un rango de valores más pequeño y finito mediante la regresión logística. A continuación, puede procesar este conjunto de datos más pequeño mediante el uso de otras técnicas de ML para obtener un análisis más preciso.

- **Visibilidad**

El análisis de regresión logística ofrece a los desarrolladores una mayor visibilidad de los procesos de software internos que otras técnicas de análisis de datos. La solución de problemas y la corrección de errores también son más fáciles porque los cálculos son menos complejos.

1.3. ¿Cuáles son las aplicaciones de la regresión logística?

La regresión logística tiene varias aplicaciones del mundo real en muchos sectores diferentes.

- **Fabricación**

Las empresas de fabricación utilizan el análisis de regresión logística para estimar la probabilidad de fallo de las piezas en la maquinaria. Luego, planifican los programas de mantenimiento en función de esta estimación para minimizar los fallos futuros.



- **Sanidad**

Los investigadores médicos planifican la atención y el tratamiento preventivos mediante la predicción de la probabilidad de enfermedad en los pacientes. Utilizan modelos de regresión logística para comparar el impacto de los antecedentes familiares o los genes en las enfermedades.

- **Finanzas**

Las empresas financieras tienen que analizar las transacciones financieras en busca de fraudes y evaluar las solicitudes de préstamos y seguros en busca de riesgos. Estos problemas son adecuados para un modelo de regresión logística porque tienen resultados discretos, como alto riesgo o bajo riesgo y fraudulento o no fraudulento.

- **Marketing**

Las herramientas de publicidad en línea utilizan el modelo de regresión logística para predecir si los usuarios harán clic en un anuncio. Como resultado, los especialistas en marketing pueden analizar las respuestas de los usuarios a diferentes palabras e imágenes y crear anuncios de alto rendimiento con los que los clientes interactuarán.

1.4. ¿Cómo funciona el análisis de regresión?

Los objetivos del bloque son:

- Entender el concepto de regresión logística.
- Conocer los casos de uso de la regresión logística.
- Aprender a interpretar los coeficientes de un modelo de regresión logística.
- Conocer la diferencia entre regresión logística y regresión lineal.

1.4.1. Definición de Regresión Logística

- La regresión logística es una técnica de modelado estadístico que se utiliza para predecir la probabilidad de que una variable dependiente categórica tome un cierto valor, basada en una o más variables independientes (predictoras).
- En otras palabras: **el objetivo de la regresión logística es modelar la probabilidad de que la variable dependiente pertenezca a una categoría particular.**

1.4.2. Modelado de probabilidad (versión simplificada)

- La **regresión logística** no solo sirve para clasificar cosas (por ejemplo, “sí” o “no”). También permite **calcular la probabilidad** de que ocurra un evento.
- El resultado que da este modelo es siempre una **probabilidad entre 0 y 1**.
- Si la probabilidad está **cerca de 1**, significa que es **muy probable** que el evento ocurra. Si está **cerca de 0**, significa que es **poco probable**.



- Para decidir a partir de qué probabilidad diremos “sí, el evento ocurrirá”, se usa un **umbral**. Por ejemplo, si ponemos un umbral de 0.5, entonces:
 - Probabilidad $\geq 0.5 \rightarrow$ predecimos que **sí** ocurrirá.
 - Probabilidad $< 0.5 \rightarrow$ predecimos que **no** ocurrirá.
- Ese umbral no es fijo: **depende del problema**. En algunos casos se usa 0.5, pero en otros puede ser más alto o más bajo.

Ejemplos:

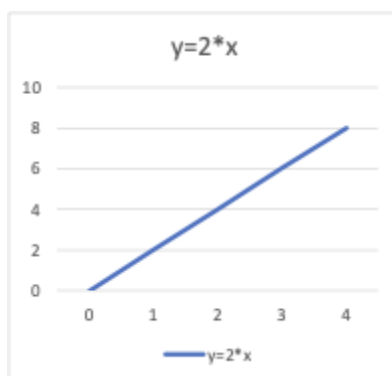
- 1) Modelar la probabilidad de que un correo sea spam de acuerdo a la frecuencia de ciertas palabras, presencia de enlaces, patrones, ...
- 2) Modelar el riesgo de cierta enfermedad de acuerdo al tipo de dieta, modo de vida, tabaquismo, ...

1.5. ¿Cómo funciona el modelo de regresión logística?

Para entender el modelo de regresión logística, primero vamos a entender las ecuaciones y las variables.

• Ecuaciones

En matemáticas, las ecuaciones dan la relación entre dos variables: x e y . Puede usar estas ecuaciones, o funciones, para trazar un gráfico a lo largo de los ejes x e y poniendo diferentes valores de x e y . Por ejemplo, si traza el gráfico para la función $y = 2 \cdot x$, obtendrá una línea recta como se muestra a continuación. Por lo tanto, esta función también se denomina función lineal.



• Variables

En estadística, las variables son los factores o atributos de datos cuyos valores varían. Para cualquier análisis, ciertas **variables son independientes o explicativas**. Estos atributos son la causa de un resultado. Otras variables son **variables dependientes o de respuesta**; sus valores dependen de las variables independientes. En general, la regresión logística explora cómo las variables independientes afectan a una variable dependiente al observar los valores de datos históricos de ambas variables.



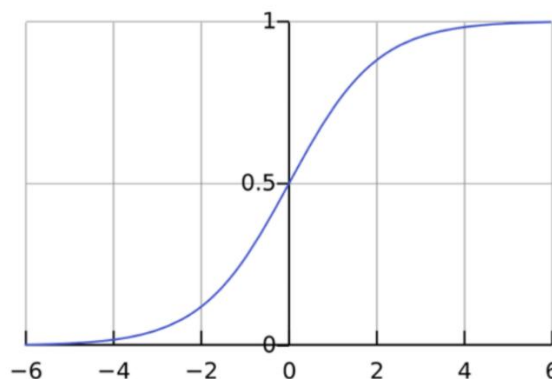
En nuestro ejemplo anterior, x se denomina variable independiente, variable predictora o variable explicativa porque tiene un valor conocido. Y se denomina variable dependiente, variable de resultado o variable de respuesta porque se desconoce su valor.

1.5.1. Función sigmoide/logística

La relación entre las probabilidades y las variables predictoras en la regresión logística se establece mediante la función logística o sigmoide, la cual transforma cualquier entrada real a un valor entre 0 y 1, haciendo que estos valores puedan interpretarse como probabilidades.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Si representamos esta ecuación de la función logística, obtendremos una curva en S como la que se muestra a continuación.



1.5.2. Deducción de la ecuación del modelo logístico

La idea de la regresión logística es muy simple: queremos calcular la probabilidad de que ocurra un evento (por ejemplo, que un correo sea spam, que un cliente compre, que un servidor falle...).

Para eso usamos n variables independientes que creemos que influyen en que ocurra o no ocurra ese evento: $x_1, x_2, \dots, x_n$

En este caso, la variable dependiente Y , sólo tomará los valores 0 o 1, y su significado es el siguiente:

$$y = \begin{cases} 1 \rightarrow \text{ocurre el evento} \\ 0 \rightarrow \text{no ocurre el evento} \end{cases}$$

Las probabilidades de que se cumpla el evento (o no se cumpla respectivamente), conociendo $x_1, x_2, \dots, x_n$, vienen dadas por los siguientes valores:

$$P(y = 1 | x_1, x_2, \dots, x_n) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}} = p$$

$$P(y = 0 | x_1, x_2, \dots, x_n) = 1 - \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}} = 1 - p$$

A partir de ellos, vamos a obtener la ecuación del modelo logístico:

$$\begin{aligned} p &= \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}} \Rightarrow 1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)} = \frac{1}{p} \Rightarrow \\ \Rightarrow e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)} &= \frac{1}{p} - 1 \Rightarrow e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)} = \frac{1 - p}{p} \Rightarrow \end{aligned}$$



$$\Rightarrow \frac{1}{e^{(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}} = \frac{1-p}{p} \Rightarrow \frac{e^{(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}}{1} = \frac{p}{1-p} \Rightarrow$$

$$\Rightarrow e^{(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)} = \frac{p}{1-p}$$

Tomando logaritmo y aplicando las propiedades de los logaritmos:

$$\ln(e^{(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}) = \ln\left(\frac{p}{1-p}\right) \Rightarrow (\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n) \ln e = \ln\left(\frac{p}{1-p}\right)$$

Así pues, la ecuación del modelo logarítmico es la siguiente:

$$e^{(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)} = \frac{p}{1-p}$$

• $\frac{p}{1-p}$ es la **razón de probabilidades (odds)**, que representa la razón de probabilidad de que ocurra el evento sobre la probabilidad de que no ocurra.

Donde:

• $\beta_0, \beta_1, \dots, \beta_k$ son los coeficientes del modelo, los cuales se estiman durante el entrenamiento del modelo y representan el efecto de las variables predictoras $X_1, X_2, \dots, X_k$ sobre el logaritmo de la razón de probabilidades.

- $\ln\left(\frac{p}{1-p}\right)$ es la función **logit** de p
- p es la probabilidad de que la variable dependiente valga 1
- $\frac{p}{1-p}$ es la razón de probabilidades (odds) que representa la razón de probabilidad de que ocurra el evento sobre la probabilidad de que no ocurra
- $\beta_0, \beta_1, \beta_2, \dots, \beta_n$ son los coeficientes del modelo, los cuales se estiman durante el entrenamiento del modelo y representan el efecto de las variables predictoras $x_1, x_2, \dots, x_n$ sobre el logaritmo de la razón de probabilidades.

1.5.3. ¿Qué son los odds?

Ejemplo:

- Supongamos que la **probabilidad** de que en unas elecciones una persona vote a un partido político A es de **0.65**.
Entonces, la probabilidad de que **no lo vote** sería $1 - 0.65 = 0.35$.
La **odd** de votar a A sobre no hacerlo sería: $odds = \frac{0.65}{0.35} = 1.86$
- Esto significa que, si tenemos una población de votantes y elegimos uno al azar, diremos que es **1.86 veces más probable que vote al partido A que no lo vote**.
En otras palabras, por cada **1.86 personas** que votan por A, habrá **1 persona** que no lo haga.
- Cuando $odds = 0.7$, significa que es **menos probable** que una persona vote por el partido A que no lo haga.
De hecho, la probabilidad de **no votar por A** sería mayor que la de votarlo.
Es decir, por cada persona que **no vota por A**, hay **0.7 personas** que **sí lo hacen**.



1.6. ¿Cuáles son los tipos de análisis de regresión logística?

Hay tres enfoques para el análisis de regresión logística basados en los resultados de la variable dependiente.

2. Regresión logística binaria

La regresión logística binaria funciona bien para problemas de clasificación binaria que solo tienen dos resultados posibles. La variable dependiente solo puede tener dos valores, como sí y no o 0 y 1.

Aunque la función logística calcula un rango de valores entre 0 y 1, el modelo de regresión binaria redondea la respuesta a los valores más cercanos. Por lo general, las respuestas por debajo de 0,5 se redondean a 0 y las respuestas por encima de 0,5 se redondean a 1, de modo que la función logística devuelve un resultado binario.

3. Regresión logística multinomial

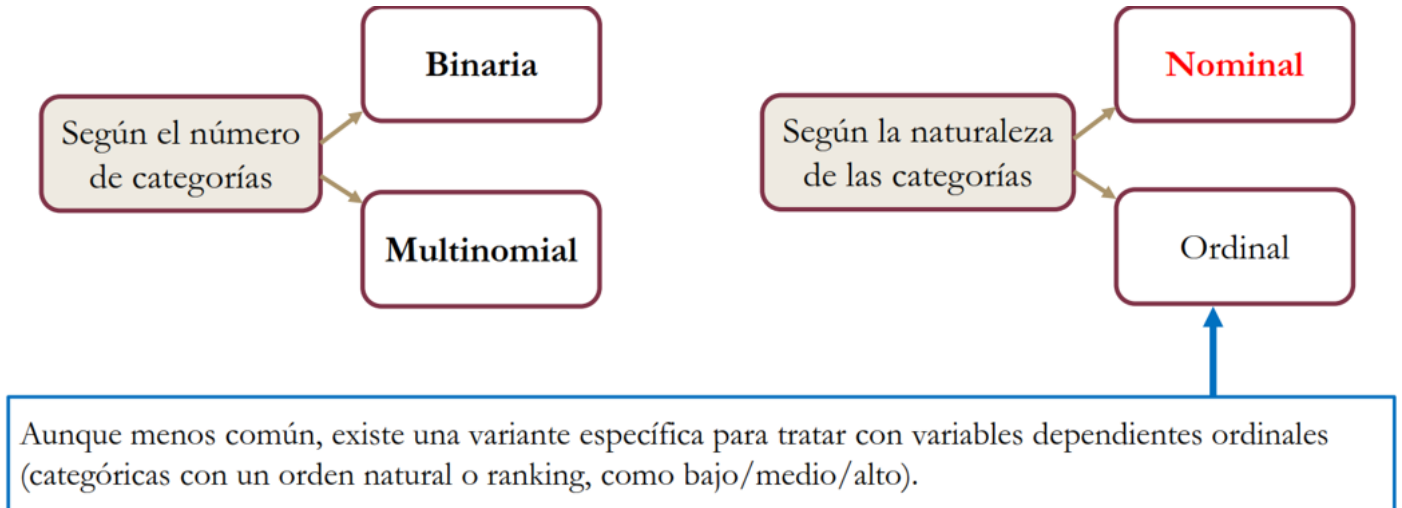
La regresión multinomial puede analizar problemas que tienen varios resultados posibles, siempre y cuando el número de resultados sea finito. Por ejemplo, puede predecir si los precios de la vivienda aumentarán un 25 %, 50 %, 75 % o 100 % en función de los datos de población, pero no puede predecir el valor exacto de una casa.

La regresión logística multinomial funciona mapeando los valores de resultado con diferentes valores entre 0 y 1. Como la función logística puede devolver un rango de datos continuos, como 0,1, 0,11, 0,12, etc., la regresión multinomial también agrupa el resultado a los valores más cercanos posibles.

4. Regresión logística ordinal

La regresión logística ordinal, o el modelo logit ordenado, es un tipo especial de regresión multinomial para problemas en los que los números representan rangos en lugar de valores reales. Por ejemplo, puede utilizar la regresión ordinal para predecir la respuesta a una pregunta de una encuesta en la que se pide a los clientes que clasifiquen su servicio como malo, regular, bueno o excelente en función de un valor numérico, como el número de artículos que le han comprado a lo largo del año.





1.7. ¿Cómo se compara la regresión logística con la regresión lineal?

Breve comparación con Regresión Lineal

Objetivo, uso y tipo de variable dependiente

- **Regresión Lineal:** Se utiliza para predecir el valor de una variable dependiente **cuantitativa** basada en una o más variables independientes.
- **Regresión Logística:** Se utiliza para predecir la **probabilidad** de que una variable dependiente **categórica** tome cierto valor discreto, basada en una o más variables independientes.

1.8. Regresión Logística con Python

1.8.2. Evaluación de un modelo de comparación: conceptos fundamentales para poder interpretar los resultados de los ejemplos en Python

Cuando entrenamos un modelo de **regresión logística** (o cualquier modelo de clasificación), necesitamos evaluar qué tan bien funciona. Para eso se usan cuatro elementos clave:

1. **Predicciones**
2. **Accuracy**
3. **Matriz de confusión**
4. **Reporte de clasificación**

1.8.2.1. Predicciones del modelo

```
y_pred = modelo.predict(X_test_scaled)
```

El modelo recibe datos nuevos (X_{test_scaled}) y devuelve una **predicción por cada ejemplo**:

- 0 → clase negativa



- 1 → clase positiva

Es simplemente lo que el modelo “cree” que va a pasar.

1.8.2.2. Accuracy (Exactitud)

```
accuracy_score(y_test, y_pred)
```

El **accuracy** es el porcentaje de predicciones correctas.

- Si accuracy = 0.90 → el modelo acierta el 90% de las veces.
- Es fácil de entender, pero **no siempre es suficiente** (por ejemplo, si las clases están muy desbalanceadas).

Es una medida general, pero no dice *qué tipo* de errores comete el modelo.

1.8.2.3. Matriz de confusión

```
confusion_matrix(y_test, y_pred)
```

La matriz de confusión muestra **cómo se distribuyen los aciertos y errores**.

	Valor Predicción 0	Valor Predicción 1
Valor Real 0	TN	FP
Valor Real 1	FN	TP

Significado:

- **TP (True Positive):** verdaderos positivos: el modelo predijo 1 y el valor real era 1 → acierto.
- **TN (True Negative):** verdaderos negativos: el modelo predijo 0 y el valor real era 0 → acierto.
- **FP (False Positive):** predijo 1 pero era 0 → error.
- **FN (False Negative):** predijo 0 pero era 1 → error.

NOTA: $TP + TN + FP + FN = N^{\circ}$ total de valores del dataset tomados para el test (15%)

Esta tabla es fundamental porque muestra **qué tipo de errores** comete el modelo.

Ejemplo práctico para informática:

- FP en detección de spam → correos buenos marcados como spam.
- FN → correos spam que pasan como buenos.

1.8.2.4. Reporte de clasificación

```
classification_report(y_test, y_pred)
```

Este reporte incluye tres métricas clave:



1. **Precision (Precisión)**

De todas las veces que el modelo dijo “es 1”, ¿cuántas eran realmente 1?

$$Precision = \frac{TP}{TP + FP}$$

- Alta precision → pocos falsos positivos.

2. **Recall (Exhaustividad o Sensibilidad)**

De todos los casos que eran realmente 1, ¿cuántos detectó el modelo?

$$Recall = \frac{TP}{TP + FN}$$

- Alto recall → pocos falsos negativos.

3. **F1-score**

Es un equilibrio entre precision y recall.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- Útil cuando las clases están desbalanceadas.
- Va de 0 a 1 (1 es perfecto).

¿Por qué existe el F1-score?

Porque a veces:

- Un modelo puede tener **mucha precision** pero **poco recall**.
- O al revés: **mucho recall** pero **poca precision**.

El F1-score penaliza los desequilibrios y da una visión más justa del rendimiento.

Interpretación sencilla

- **F1 = 1** → modelo perfecto.
- **F1 alto (0.8–1)** → buen equilibrio entre aciertos positivos y detección de casos reales.
- **F1 medio (0.5–0.7)** → el modelo falla bastante en algún tipo de error.
- **F1 bajo (< 0.5)** → el modelo no está funcionando bien.

Ejemplo: En detección de spam:



- **Alta precision pero bajo recall**
→ casi todos los correos marcados como spam lo son,
→ pero se escapan muchos correos spam reales.
- **Alto recall pero baja precision**
→ detecta casi todo el spam,
→ pero mete muchos correos buenos en la carpeta de spam.

El **F1-score** INFORMA SOBRE si el modelo está equilibrado o si está “cojeando” por algún lado.

1.8.2. Ejemplo1 en Python: Predicción de compra en tienda online

Supongamos que queremos predecir si un visitante de una tienda online realizará una compra (Sí/No) basándonos en dos características: el tiempo que pasa en el sitio (en minutos) y la cantidad de productos que añade al carrito. Para ello, generaremos un conjunto de datos sintético con estas características y la variable objetivo "compra" que indica si el visitante compró o no.

```
import numpy as np
import pandas as pd

# Semilla para reproducibilidad
np.random.seed(42)

# Número de muestras
n_samples = 5000

# Generar tiempo en sitio (minutos) entre 1 y 10
tiempo = np.random.uniform(1, 10, n_samples)

# Generar cantidad de productos en carrito entre 0 y 5
productos = np.random.randint(0, 6, n_samples)

# Regla para determinar compra:
# Si tiempo > 5 y productos > 2 → compra = 1, si no → 0
compra = ((tiempo > 5) & (productos > 2)).astype(int)

# Crear DataFrame
data = pd.DataFrame({
    'tiempo': tiempo,
    'productos': productos,
    'compra': compra
})

print(data.head())
print("\nTamaño del dataset:", data.shape)
```

1. Preparación del entorno en Python

```
import pandas as pd
import numpy as np
```



```
# Librerías para crear el modelo de regresión logística
from sklearn import linear_model, model_selection
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Librerías para visualización
import matplotlib.pyplot as plt
import seaborn as sb

# Configuración para mostrar gráficos dentro del notebook
%matplotlib inline

# Comentario:
# Este bloque prepara todas las herramientas necesarias para:
# - Cargar y manipular datos (pandas, numpy)
# - Entrenar un modelo de regresión logística (sklearn)
# - Evaluar el modelo (accuracy, matriz de confusión, reporte de clasificación)
# - Visualizar datos y resultados (matplotlib, seaborn)
```

2. Carga del dataset generado

Antes de entrenar el modelo, necesitamos cargar el dataset que generado previamente.

```
# Si ya tienes el DataFrame 'data' generado en memoria, no necesitas hacer nada.
# Si lo guardaste en un archivo CSV, descomenta la siguiente línea:
# data = pd.read_csv("dataset_tienda_online.csv")

# Mostrar primeras filas
data.head()
```

3. División Train/Test

Dividimos los datos en entrenamiento (80%) y prueba (20%). Esto permite evaluar el modelo con datos que nunca ha visto.

```
# DIVISIÓN TRAIN/TEST
X = data[['tiempo', 'productos']] # Variables predictoras
y = data['compra']               # Variable objetivo

# División 80% entrenamiento, 20% prueba
X_train, X_test, y_train, y_test = model_selection.train_test_split(
    X, y, test_size=0.2, random_state=42
)
X_train.shape, X_test.shape
```

4. Entrenamiento del modelo de regresión logística

Creamos el modelo, lo entrenamos y mostramos los coeficientes aprendidos.



ENTRENAMIENTO DEL MODELO

```
modelo = linear_model.LogisticRegression()
modelo.fit(X_train, y_train)

print("Coeficientes del modelo:", modelo.coef_)
print("Intercept:", modelo.intercept_)
```

5. Evaluación completa del modelo

Predicciones

```
# Predicciones
y_pred = modelo.predict(X_test)

# Métricas
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nMatriz de confusión:\n", confusion_matrix(y_test, y_pred))
print("\nReporte de clasificación:\n", classification_report(y_test, y_pred))
```

Visualización de la frontera de decisión

Esto permite ver cómo el modelo separa las clases en el plano (tiempo vs productos). Es muy útil para interpretar modelos simples.

VISUALIZACIÓN DE LA FRONTERA DE DECISIÓN

```
# Crear una malla de puntos
x_min, x_max = X['tiempo'].min() - 1, X['tiempo'].max() + 1
y_min, y_max = X['productos'].min() - 1, X['productos'].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

# Crear DataFrame con nombres de columnas para evitar el warning
grid = pd.DataFrame({
    'tiempo': xx.ravel(),
    'productos': yy.ravel()
})

# Predicción sobre la malla
Z = modelo.predict(grid)
Z = Z.reshape(xx.shape)

# Gráfico
plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
```



```
# Puntos reales
sb.scatterplot(data=data, x='tiempo', y='productos', hue='compra', palette='coolwarm')

plt.title("Frontera de decisión - Regresión Logística")
plt.xlabel("Tiempo en sitio (min)")
plt.ylabel("Productos añadidos")
plt.show()
```

1.8.3. Ejemplo2 en Python: Predicción SO (regresión logística multinomial)

Queremos predecir el **Sistema Operativo (SO)** que utiliza un usuario cuando accede a una página web. Las clases posibles son:

- **0** → **Windows**
- **1** → **Macintosh**
- **2** → **Linux**

Para ello, mediremos **cuatro variables numéricas** que un servidor web puede registrar automáticamente y que influyen en el tipo de dispositivo y sistema operativo del usuario:

Variables numéricas utilizadas

1. **ancho_pantalla** (en píxeles)
 - Windows: muy variable
 - macOS: resoluciones altas y estables
 - Linux: resoluciones medias típicas de monitores estándar
2. **alto_pantalla** (en píxeles)
 - Misma lógica que el ancho
3. **velocidad_scroll** (píxeles por segundo)
 - macOS: scroll suave y acelerado
 - Windows: scroll segmentado
 - Linux: scroll más lineal
4. **tiempo_carga_ms** (tiempo de carga de la página en milisegundos)
 - Windows: rendimiento medio
 - macOS: optimización alta → tiempos menores
 - Linux: depende del entorno → tiempos variables

A partir de estas variables, generaremos un dataset sintético de **5000 usuarios**, simulando patrones realistas.

CÓDIGO COMPLETO PARA GENERAR EL DATASET (5000 registros)

Incluye la exportación a CSV con nombre **SO.csv**.

```
import numpy as np
import pandas as pd

# Semilla para reproducibilidad
np.random.seed(42)

# Número de muestras
n = 5000

# -----
# 1. Generación de variables numéricas
# -----
```



```
# Ancho de pantalla (px)
# Windows: muy variado, macOS: alto, Linux: medio
ancho_windows = np.random.normal(1366, 200, int(n*0.6))
ancho_mac = np.random.normal(2560, 150, int(n*0.25))
ancho_linux = np.random.normal(1920, 180, int(n*0.15))

ancho_pantalla = np.concatenate([ancho_windows, ancho_mac, ancho_linux])

# Alto de pantalla (px)
alto_windows = np.random.normal(768, 120, int(n*0.6))
alto_mac = np.random.normal(1600, 100, int(n*0.25))
alto_linux = np.random.normal(1080, 130, int(n*0.15))

alto_pantalla = np.concatenate([alto_windows, alto_mac, alto_linux])

# Velocidad de scroll (px/s)
scroll_windows = np.random.normal(800, 150, int(n*0.6))
scroll_mac = np.random.normal(1200, 200, int(n*0.25))
scroll_linux = np.random.normal(900, 180, int(n*0.15))

velocidad_scroll = np.concatenate([scroll_windows, scroll_mac, scroll_linux])

# Tiempo de carga (ms)
carga_windows = np.random.normal(900, 150, int(n*0.6))
carga_mac = np.random.normal(700, 120, int(n*0.25))
carga_linux = np.random.normal(850, 140, int(n*0.15))

tiempo_carga_ms = np.concatenate([carga_windows, carga_mac, carga_linux])

# -----
# 2. Variable objetivo (SO)
# -----
# 0 = Windows, 1 = Macintosh, 2 = Linux
so = np.array(
    [0]*int(n*0.6) +
    [1]*int(n*0.25) +
    [2]*int(n*0.15)
)

# -----
# 3. Crear DataFrame
# -----
data = pd.DataFrame({
    'ancho_pantalla': ancho_pantalla,
    'alto_pantalla': alto_pantalla,
    'velocidad_scroll': velocidad_scroll,
    'tiempo_carga_ms': tiempo_carga_ms,
    'so': so
})

# -----
```



```
# 4. Exportar a CSV
# -----
data.to_csv("SO.csv", index=False)

print("Dataset generado con éxito. Tamaño:", data.shape)
print(data.head())
```

Tabla resumen, mostrando los **niveles numéricos utilizados para discriminar cada Sistema Operativo** en el dataset sintético.

Variable numérica	Windows (0)	Macintosh (1)	Linux (2)	Interpretación
Ancho de pantalla (px)	Normal(1366, 200)	Normal(2560, 150)	Normal(1920, 180)	macOS usa pantallas Retina; Windows es muy variable; Linux suele usar monitores estándar.
Alto de pantalla (px)	Normal(768, 120)	Normal(1600, 100)	Normal(1080, 130)	macOS tiene resoluciones altas; Windows y Linux muestran valores más bajos.
Velocidad de scroll (px/s)	Normal(800, 150)	Normal(1200, 200)	Normal(900, 180)	macOS tiene scroll más suave y rápido; Linux intermedio; Windows más segmentado.
Tiempo de carga (ms)	Normal(900, 150)	Normal(700, 120)	Normal(850, 140)	macOS carga más rápido; Windows más lento; Linux depende del entorno.

1. Preparación del entorno

```
# Librerías para manejo de datos
import pandas as pd
import numpy as np

# Librerías para machine learning
from sklearn import linear_model, model_selection
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Visualización
import matplotlib.pyplot as plt
import seaborn as sb
%matplotlib inline
```

2. Carga del dataset SO.csv

```
# CARGA DEL DATASET
data = pd.read_csv("SO.csv")

# Mostrar primeras filas
data.head()
```

3. Exploración inicial del dataset

```
# EXPLORACIÓN INICIAL
```



```
print(data.shape)
print(data.describe())
print(data['so'].value_counts())
sb.pairplot(data, hue='so')
```

4. División Train/Test

```
# DIVISIÓN TRAIN/TEST

X = data[['ancho_pantalla', 'alto_pantalla', 'velocidad_scroll', 'tiempo_carga_ms']]
y = data['so']

X_train, X_test, y_train, y_test = model_selection.train_test_split(
    X, y, test_size=0.2, random_state=42
)

X_train.shape, X_test.shape
```

5. Entrenamiento del modelo (Regresión Logística Multinomial)

```
# ESCALADO DE VARIABLES
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ENTRENAMIENTO DEL MODELO MULTINOMIAL (corregido)
modelo = linear_model.LogisticRegression(
    solver='lbfgs',
    max_iter=1000
)

modelo.fit(X_train_scaled, y_train)

print("Coeficientes del modelo:")
print(modelo.coef_)
print("\nIntercept:")
print(modelo.intercept_)
```

6. Evaluación completa del modelo

```
# EVALUACIÓN COMPLETA DEL MODELO

# 1. Predicciones usando los datos escalados
y_pred = modelo.predict(X_test_scaled)

# 2. Accuracy
print("Accuracy del modelo:", accuracy_score(y_test, y_pred))
```



```
# 3. Matriz de confusión
print("\nMatriz de confusión:")
print(confusion_matrix(y_test, y_pred))

# 4. Reporte de clasificación
print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred))
```

7. Visualización de la frontera de decisión (solo 2 variables)

Para visualizar una frontera, debemos reducir a 2 variables.

Usaremos:

- ancho_pantalla
- velocidad_scroll

```
# 🧠 VISUALIZACIÓN DE LA FRONTERA DE DECISIÓN (2 variables)

from sklearn.preprocessing import StandardScaler

# Seleccionamos solo dos variables para poder graficar
X2 = data[['ancho_pantalla', 'velocidad_scroll']]
y2 = data['so']

# Escalado de estas dos variables
scaler2 = StandardScaler()
X2_scaled = scaler2.fit_transform(X2)

# Entrenamos un modelo SOLO con estas dos variables
modelo2 = linear_model.LogisticRegression(
    solver='lbfgs',
    max_iter=1000
)
modelo2.fit(X2_scaled, y2)

# Crear malla de puntos para graficar la frontera
x_min, x_max = X2['ancho_pantalla'].min() - 50, X2['ancho_pantalla'].max() + 50
y_min, y_max = X2['velocidad_scroll'].min() - 50, X2['velocidad_scroll'].max() + 50

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 300),
    np.linspace(y_min, y_max, 300)
)

# Convertimos la malla en DataFrame
grid = pd.DataFrame({
    'ancho_pantalla': xx.ravel(),
    'velocidad_scroll': yy.ravel()
})

# Escalamos la malla igual que los datos reales
grid_scaled = scaler2.transform(grid)
```



```
# Predicción sobre la malla
Z = modelo2.predict(grid_scaled)
Z = Z.reshape(xx.shape)

# Gráfico
plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')

# Puntos reales
sb.scatterplot(
    data=data,
    x='ancho_pantalla',
    y='velocidad_scroll',
    hue='so',
    palette='coolwarm'
)

plt.title("Frontera de decisión - Regresión Logística Multinomial")
plt.xlabel("Ancho de pantalla (px)")
plt.ylabel("Velocidad de scroll (px/s)")
plt.show()
```

8. Interpretación del modelo

```
# INTERPRETACIÓN DE COEFICIENTES
```

```
coef_df = pd.DataFrame(modelo.coef_, columns=X.columns)
coef_df['Clase (SO)'] = ['Windows (0)', 'Macintosh (1)', 'Linux (2)']
coef_df
```

BIBLIOGRAFÍA

1. Fundamentos del Análisis de Datos con Python: <https://www.quantum.tech/es/learning-path/ai-engineering>

2. Curso introductorio en Google Colab (gratuito y práctico)

Un curso abierto creado por estudiantes del IEEE, ideal para empezar a manipular datos, automatizar tareas y visualizar información.

https://colab.research.google.com/github/IEEESBITBA/Curso-python/blob/master/Curso_Analisis_de_Datos_Clases/Clase_1_Analisis_de_Datos.ipynb#scrollTo=0006-eqnUur

3. Regresión Logística

<https://aws.amazon.com/es/what-is/logistic-regression/>



https://es.wikipedia.org/wiki/Regresi%C3%B3n_log%C3%ADstica

4. Curso de Análisis de Datos Gratis (Python, Excel y R)

Curso certificado que cubre fundamentos del análisis de datos, limpieza, preparación y técnicas estadísticas básicas.

<https://videocursos.co/curso-de-analisis-de-datos-con-python/>

5. Repositorio “Awesome Data Analysis” (GitHub)

Colección de más de 500 recursos gratuitos sobre análisis de datos, Python, estadística y visualización.

<https://github.com/PavelGrigoryevDS/awesome-data-analysis>

